

Homework rules

- Group discussion is *highly recommended*. But you have to write down your solution by *yourself*. Aim for clarity and conciseness in your solution, emphasizing the main ideas over low-level details.
- You should *cite* all the references that help you to solve the homework, including the people you have discussed with and the online material you looked at.
- You are required to type your solution using latex and submit a *pdf* to Gradescope and a *tex* file to NTUCool. You may type your solution in English (preferable) or Chinese.
- You are not expected to solve *all* problems. Just try your best! If you don't solve a problem, try your best to write down your thinking process.
- You may use AI however you wish to deepen your understanding of the lecture material. Upload the notes, talk to your AI about them, ask for more explanation or examples; it's all fine. You may not use LLMs in any way to work on your homework. You may not upload assignments, ask for hints, ask how certain concepts from the lectures might be applied to specific homework problems, or upload your assignments to check for correctness or clarity or anything else. You may not include any AI generated content whatsoever in your homework submissions (This policy is taken from MIT 6.5620).
- We will use a *random-checking policy* in which students may be asked to present their homework solutions during TA office hours, with the expectation that everyone will present *at least once*. Evaluation is based on *consistency*, meaning students must be able to reproduce the solutions they originally submitted. If a student fails this consistency check, they will receive a significant *percentage deduction* from their homework score.
- 作業總分為44分，可得最大計分為38分。
- Deadline 5/31(日)23:59，遲交時間最晚6/1(二) 23:59 (遲交成績打9折)，超過則以0分計算。

1. (8 points, Broken IND-CPA to IND-CCA transformation)

Definition. A MAC system is a pair of PPT algorithms Sign , a signing algorithm, and Vrfy , a verification algorithm.

- Sign takes as input a key k and a message m and output a tag tag .
- Vrfy is a deterministic algorithm that takes as input a key k , a message m and a tag tag and outputs accept or reject.

We require that tags generated by Sign are always accepted by Vrfy ; that is, the MAC must satisfy the following correctness property: for all keys k and for all message m ,

$$\Pr[\text{Vrfy}_k(m, \text{Sign}_k(m)) = \text{accept}] = 1.$$

As usual, We denote by $\mathcal{K}$ the key space, $\mathcal{M}$ the message space and $\mathcal{T}$ the tag space.

Definition. (MAC security with verification queries) For a given MAC system $\mathcal{I} = (\text{Sign}, \text{Vrfy})$, the MAC experiment runs as follows:

- The challenger picks a random $k \leftarrow \mathcal{K}$;
- Adv queries the challenger several times. For $i = 1, 2, \dots$, the i th signing query is a message $m_i \in \mathcal{M}$.
- Given m_i the challenger computes a tag $t_i \leftarrow \text{Sign}_k(m_i)$ and returns t_i to Adv .
- Eventually Adv outputs a candidate forgery pair $(m, t) \in \mathcal{M} \times \mathcal{T}$ that is not among the signed pairs, i.e.,

$$(m, t) \notin \{(m_1, t_1), (m_2, t_2), \dots\}.$$

- (e) **Adv** wins iff the challenger ever responds to a verification query with accept, i.e., $\text{Vrfy}_k(m, t) = \text{accept}$.

Definition. We say that a MAC system $\mathcal{I}$ is secure if for all PPT adversary **Adv**, there is a negligible function $\mu(\cdot)$ such that in the above experiment MAC

$$\Pr[\mathbf{Adv} \text{ wins}] \leq \mu(n).$$

Consider the following attempt at transformation a CPA-secure scheme to a IND-CCA-secure one. Let $(\text{KeyGen}, \text{Enc}, \text{Dec})$ be a IND-CPA-secure PKE defined over $(\mathcal{K} \times \mathcal{M}, \mathcal{C})$ and let $(\text{Sign}, \text{Vrfy})$ be a secure MAC with key space $\mathcal{K}$. We construction a new PKE scheme $\text{PKE}' = (\text{KeyGen}', \text{Enc}', \text{Dec}')$ with message space $\mathcal{M}$, as follows:

- (a) $\text{KeyGen}' = \text{KeyGen}$;
- (b) $\text{Enc}'_{pk}(m)$:
 - i. $k \leftarrow \mathcal{K}$;
 - ii. $ct \leftarrow \text{Enc}_{pk}(k, m)$;
 - iii. $t \leftarrow \text{Sign}_k(ct)$;
 - iv. output $ct' = (ct, t)$.
- (c) $\text{Dec}'_{sk}(ct')$:
 - i. $(k, m) \leftarrow \text{Dec}_{sk}(ct)$;
 - ii. if $\text{Vrfy}_k(ct, t) = \text{accept}$, output m ;
 - iii. otherwise, output $\perp$.

One might expect this scheme to be IND-CCA-secure because a change to a ciphertext (ct, t) will invalidate the MAC tag t . Show that this is INCORRECT. i.e., show a IND-CPA-secure PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ for which $\text{PKE}' = (\text{KeyGen}', \text{Enc}', \text{Dec}')$ is not IND-CCA-secure (for any choice of MAC).

2. **(12 points, Various FO transformation)** In this problem, let $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be a PKE scheme.

- (a) (4 points) Suppose that Π is δ -correct, γ -spread and OW-CPA secure. Let $G : \mathcal{M} \rightarrow \mathcal{R}$ be a random oracle and T be the transformation defined in the class. Show that $T[\Pi, G]$ is OW-CCA secure.

In the following we introduce another variant U^χ of the U transformation taught in the class that converts a OW-CCA PKE scheme into an IND-CCA KEM. The transformation U^χ is used in CRYSTALS-Kyber, the NIST PQC standard KEM, works similarly to the U transformation defined in the class but with “implicit rejection” of inconsistent ciphertexts. The construction is given as follows: the transformation U^χ takes as input a PKE scheme $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ and an RO $H : \mathcal{M} \times \mathcal{C} \rightarrow \mathcal{K}$, and outputs a KEM $\Pi_{\text{KEM}^\chi} = U^\chi[\Pi, H] = (\text{KeyGen}^\chi, \text{Encaps}, \text{Decaps}^\chi)$. The algorithms are defined below:

$\text{KeyGen}^\mathcal{X}(1^n)$	$\text{Encaps}(\text{pk}')$	$\text{Decaps}_{\text{sk}}^\mathcal{X}(\text{ct})$
1 : $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^n)$	1 : $m \xleftarrow{\$} \mathcal{M}$	1 : Parse $\text{sk}' = (\text{sk}, s)$
2 : $s \xleftarrow{\$} \mathcal{M}$	2 : $\text{ct} \leftarrow \text{Enc}_{\text{pk}}(m)$	2 : $m' = \text{Dec}_{\text{sk}}(\text{ct})$
3 : $\text{sk}' := (\text{sk}, s)$	3 : $k := H(m, \text{ct})$	3 : if $m' \neq \perp$
4 : return $(\text{pk}' := \text{pk}, \text{sk}')$	4 : return (k, ct)	4 : return $k := H(m', \text{ct})$
		5 : else return $k := H(s, \text{ct})$

One can see that the Encaps is the same as the one defined in the class. The $\mathcal{U}^\mathcal{X}$ and $\mathcal{U}$ essentially differ in decapsulation: Decaps from $\mathcal{U}$ rejects if ct decrypts to $\perp$, whereas Decaps $^\mathcal{X}$ from $\mathcal{U}^\mathcal{X}$ returns a pseudorandom key k .

- (b) (8 points) Show that if $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec})$ is a OW-CCA secure PKE scheme and $H : \mathcal{M} \times \mathcal{C} \rightarrow \mathcal{K}$ is an RO, then $\Pi_{\text{KEM}^\mathcal{X}} = \mathcal{U}^\mathcal{X}[\Pi, H]$ is IND-CCA secure.

Remark. Actually both $\mathcal{U}$ and $\mathcal{U}^\mathcal{X}$ can take as input a OW-CCA PKE with randomized encryption and output an IND-CCA KEM. However there are also other variants, for instances, $\mathcal{U}_m$ and $\mathcal{U}_m^\mathcal{X}$, whose output KEMs derive key as $k = H(m)$, require that the input PKE is deterministic as the output of T transformation.

3. (12 points, Bit commitment in the RO model)

In this problem, we will present a concrete definition of non-interactive commitment schemes in the RO model and a simple construction, and you are asked to prove its security.

Let $n \in \mathbb{N}$ be the security parameter and $\text{RO} : \{0, 1\}^{n+1} \rightarrow \{0, 1\}^{3n}$ be a random oracle. A non-interactive bit commitment scheme is simply an oracle algorithm Com^{RO} that on input a bit $b \in \{0, 1\}$ and a randomness $r \in \{0, 1\}^n$, outputs a commitment $\text{com} = \text{Com}^{\text{RO}}(b; r)$. A committer C can commit to a bit $b \in \{0, 1\}$ to a receiver R by sampling a uniformly random $r \xleftarrow{\$} \{0, 1\}^n$ and computing $\text{com} = \text{Com}^{\text{RO}}(b; r)$. C can later open the commitment com and reveal b by sending b and r to R , who can verify the opening by checking if $\text{com} = \text{Com}^{\text{RO}}(b; r)$.

Consider the following simple construction $\text{Com}^{\text{RO}}(b; r) := \text{RO}(b|r)$. Show that this simple construction satisfies the following binding and hiding properties.

- (a) (3 points) Hiding: Consider the following hiding security game:

- The challenger samples $b \xleftarrow{\$} \{0, 1\}$ and $r \xleftarrow{\$} \{0, 1\}^n$ uniformly at random, computes and sends $\text{com} = \text{Com}^{\text{RO}}(b; r)$ to the adversary.
- Upon receiving com , the adversary outputs a guess $b' \leftarrow \text{Adv}^{\text{RO}}(\text{com})$.
- The adversary wins if $b = b'$.

Show that for any oracle adversary Adv^{RO} that makes at most q -queries,

$$\Pr[\text{Adv}^{\text{RO}} \text{ wins}] \leq \frac{1}{2} + \frac{2q}{2^n}$$

- (b) (3 points) Binding: Show that for any oracle adversary Adv^{RO} that makes at most q -queries,

$$\Pr[\text{com} = \text{Com}^{\text{RO}}(0; r_0) = \text{Com}^{\text{RO}}(1; r_1) : (\text{com}, r_0, r_1) \leftarrow \text{Adv}^{\text{RO}}()] \leq \frac{1}{2^{3n}} + \frac{q^2}{2^{3n}}.$$

(c) (6 points) In HW2, we also see a construction of a bit commitment scheme from PRG. They are the same primitive, but defined in different models (one in the plain model and the other in the random oracle model). This is common in theoretical cryptography. In this problem, you are asked to compare two of them.

- (2 points) Compare the syntax, security, and hiding definitions.
- (2 points) Compare how both constructions achieve the hiding and binding properties.
- (2 points) Discuss why, in the random oracle model, it is possible to obtain a non-interactive commitment scheme.

4. (8 points, From KEM to PKE) In class, we've shown how to achieve an IND-CCA secure KEM via FO transformation. In this problem, we show that we can upgrade it to an IND-CCA secure PKE with the help of an IND-CCA secure SKE scheme.

Let us first recall the definition of a symmetric-key encryption scheme from HW3.

Definition. A symmetric-key encryption scheme (SKE scheme) is a triple $(\text{KeyGen}, \text{Enc}, \text{Dec})$ of PPT algorithms with the following syntax:

- $\text{KeyGen}(1^n)$ takes as input the security parameter 1^n and outputs a key k .
- $\text{Enc}_k(m)$ takes as inputs a key k and a message $m \in \mathcal{M}_n$, where $\mathcal{M}_n$ is the space of messages, and outputs a ciphertext $ct \in \mathcal{C}_n$.
- $\text{Dec}_k(ct)$ takes as inputs a key k and a ciphertext ct and outputs a message m or $\perp$ denoting failure.

It is required to satisfy the following correctness property:

- (Correctness) There exists a negligible function $\mu(\cdot)$ such that for every $n \in \mathbb{N}$ and every message $m \in \mathcal{M}_n$,

$$\Pr[k \leftarrow \text{KeyGen}(1^n) : \text{Dec}_k(\text{Enc}_k(m)) = m] \geq 1 - \mu(n).$$

Here we require the CCA security of SKE. Consider the following CCA experiment SKE^{CCA} :

- $k \leftarrow \text{KeyGen}(1^n)$;
- The adversary Adv is given as input 1^n and the oracle access $\text{Enc}_k(\cdot)$ and $\text{Dec}_k(\cdot)$, and outputs a pair of messages m_0, m_1 ;
- A bit $b \leftarrow \{0, 1\}$ is uniformly chosen at random and a ciphertext $ct \leftarrow \text{Enc}_k(m_b)$ is computed and is given to the adversary Adv ;
- The adversary Adv continues to have oracle access to $\text{Enc}_k(\cdot)$ and $\text{Dec}_k(\cdot)$ except that $\text{Dec}_k(ct)$ now returns $\perp$, i.e. Adv is not allowed to query the challenge ciphertext. At last, Adv outputs a bit b' ;
- Adv wins iff $b' = b$.

An SKE scheme $(\text{KeyGen}, \text{Enc}, \text{Dec})$ is CCA-secure if for all PPT adversaries Adv there is a negligible function $\mu(\cdot)$ such that, for any $n \in \mathbb{N}$, in the CCA experiment SKE^{CCA} we have

$$\Pr[\text{Adv wins}] \leq \frac{1}{2} + \mu(n)$$

where the probability is taken over the randomness used by Adv as well as used in the experiment SKE^{CCA} .

Now, we show how to obtain an IND-CCA secure PKE scheme PKE via :

- An IND-CCA secure KEM scheme $\text{KEM} = (\text{KeyGen}_{\text{KEM}}, \text{Encaps}, \text{Decaps})$, where the output key $k \in \{0, 1\}^{l(n)}$ for security parameter 1^n
- An IND-CCA secure SKE scheme $\text{SKE} = (\text{KeyGen}_{\text{SKE}}, \text{Enc}_{\text{SKE}}, \text{Dec}_{\text{SKE}})$, where $\text{KeyGen}_{\text{SKE}}(1^n)$ uniformly samples $k \leftarrow \{0, 1\}^{l(n)}$. The message space on security parameter 1^n is defined as $\mathcal{M}_n$.

The combined PKE scheme $\text{PKE} = (\text{KeyGen}_{\text{PKE}}, \text{Enc}_{\text{PKE}}, \text{Dec}_{\text{PKE}})$ with message space also $\mathcal{M}_n$ is defined as follow:

- $\text{KeyGen}_{\text{PKE}}(1^n)$: Runs $(pk, sk) \leftarrow \text{KeyGen}_{\text{KEM}}(1^n)$ and returns (pk, sk) .
- $\text{Enc}_{\text{PKE}}(pk, m \in \mathcal{M}_n)$:
 - $(ct_{\text{KEM}}, k) \leftarrow \text{Encaps}(pk)$
 - $ct_{\text{SKE}} = \text{Enc}_{\text{SKE}}(k, m)$
 - return $ct_{\text{PKE}} = (ct_{\text{KEM}}, ct_{\text{SKE}})$
- $\text{Dec}_{\text{PKE}}(sk, ct_{\text{PKE}} = (ct_{\text{KEM}}^*, ct_{\text{SKE}}^*))$:
 - $k^* \leftarrow \text{Decaps}(sk, ct_{\text{KEM}}^*)$, return $\perp$ if $k^* = \perp$
 - otherwise, return $m^* \leftarrow \text{Dec}_{\text{SKE}}(k^*, ct_{\text{SKE}}^*)$

Show that the previous construction is indeed an IND-CCA secure PKE scheme.

(Hint. In order to play with the PKE adversary, you will need to simulate the decryption oracle. It is natural to see that you will need the decryption oracle from the SKE scheme. How can you bridge the two interface? To achieve this, you will have to somehow make use of the IND-CCA property of the KEM scheme.)

5. (4 points, OR-Sigma protocol is secure) Suppose that we have two Sigma protocols (P_0, V_0) for $\mathcal{R}_0 \subset \mathcal{X}_0 \times \mathcal{Y}_0$ and (P_1, V_1) for $\mathcal{R}_1 \subset \mathcal{X}_1 \times \mathcal{Y}_1$. We assume that

- Both Sigma protocols use the same challenge space $\mathcal{C} = \{0, 1\}^n$.
- Both protocols are HVZK with simulators Sim_0 and Sim_1 respectively.

We can combine them to form a Sigma protocol for the relation $\mathcal{R}_{\text{OR}} \subset \mathcal{X}_{\text{OR}} \times \mathcal{Y}_{\text{OR}}$, where $\mathcal{X}_{\text{OR}} = \mathcal{X}_0 \times \mathcal{X}_1$, and $\mathcal{Y}_{\text{OR}} = \{0, 1\} \times (\mathcal{Y}_0 \cup \mathcal{Y}_1)$

$$\mathcal{R}_{\text{OR}} = \{((x_0, x_1), (b, y)) \in ((\mathcal{X}_0 \times \mathcal{X}_1) \times (\{0, 1\} \times (\mathcal{Y}_0 \cup \mathcal{Y}_1))) \mid (x_b, y) \in \mathcal{R}_b\}$$

In other words, for a given pair of statements $x_0 \in \mathcal{X}_0$ and $x_1 \in \mathcal{X}_1$, this OR protocol allows a prover to convince a skeptical verifier that he “knows” a witness for x_0 or a witness for x_1 . Nothing else should be revealed. In particular the protocol should not reveal which witness the prover actually have. The protocol (P, V) runs as follows, where the prover P is initialized with $((x_0, x_1), (b, y)) \in \mathcal{R}_{\text{OR}}$, the verifier V is initialized with $(x_0, x_1) \in \mathcal{X}_0 \times \mathcal{X}_1$, and $d := 1 - b$:

- P computes $(\text{com}_d, \text{ch}_d, \text{resp}_d) \leftarrow \text{Sim}_d(x_d)$. P also runs $P_b(y, x_b)$ to get a commitment com_b and sends the pair $(\text{com}_0, \text{com}_1)$ to V ;
- V computes $\text{ch} \leftarrow \mathcal{C}$ and sends the challenge ch to P ;
- P computes $\text{ch}_b \leftarrow \text{ch} \oplus \text{ch}_d$. P feeds the challenge ch_b to $P_b(y, x_b)$, obtaining a response resp_b , and sends $(\text{ch}_0, \text{resp}_0, \text{resp}_1)$ to V ;
- V computes $\text{ch}_1 \leftarrow \text{ch} \oplus \text{ch}_0$ and checks that $(\text{com}_i, \text{ch}_i, \text{resp}_i)$ is an accepting conversation for x_i , where $i = 0, 1$.

Show that the above protocol is indeed a Sigma protocol for $\mathcal{R}_{OR}$. In other word, show that the protocol satisfies completeness, special soundness, and HVZK.